

Route Optimization Visualizer (Streamlit)

Create a complete, production-quality Python project called **Route Optimization Visualizer**. The application should be built using **Streamlit** and designed to be deployed publicly on **Streamlit Community Cloud**.

Goal

Build an interactive web application that allows users to create or load weighted graphs and visualize how different pathfinding algorithms find routes between two nodes.

The application should prioritize clean code, modular architecture, and an attractive user interface suitable for a software engineering portfolio.

Tech Stack

- Python 3.12+
 - Streamlit
 - NetworkX
 - Plotly (preferred) or Matplotlib
 - NumPy
 - Pandas (if needed)
-

Project Structure

Organize the project into multiple files instead of placing everything into one script.

Example:

```
route-optimization-visualizer/  
|  
├─ app.py  
├─ requirements.txt  
├─ README.md  
|  
├─ algorithms/  
|   ├── bfs.py  
|   ├── dfs.py  
|   ├── dijkstra.py  
|   ├── astar.py  
|   └─ helpers.py
```

```
|
|
|─ graph/
|   |─ graph_generator.py
|   |─ graph_loader.py
|   └─ visualization.py
|
|─ utils/
|   |─ metrics.py
|   └─ session.py
|
|─ assets/
|   └─ screenshots/
|
|─ data/
```

Keep functions small and well documented.

Use type hints throughout the project.

Follow PEP8 formatting.

Features

Graph Creation

Allow users to

- Generate a random weighted graph
- Specify
 - number of nodes
 - edge probability
 - minimum edge weight
 - maximum edge weight
- Create custom graphs manually
- Upload graphs from CSV

CSV format

```
source,target,weight
A,B,4
A,C,2
C,D,5
```

Algorithms

Implement the following from scratch.

Do NOT call built-in shortest path implementations.

Breadth-First Search

For unweighted graphs.

Show

- traversal order
- discovered nodes
- final path

Depth-First Search

Show

- traversal order
- recursion stack (or simulated stack)
- final path if found

Dijkstra

Use a priority queue.

Display

- current distances
- visited nodes
- predecessor table

A*

Allow users to assign x/y coordinates to nodes.

Use Euclidean distance as the heuristic.

Display

- f score
 - g score
 - heuristic value
 - open set
 - closed set
-

Visualization

This should be one of the strongest parts of the project.

Draw the graph interactively.

Show

- node labels
- edge weights
- highlighted shortest path
- current node
- explored nodes
- frontier

Use different colors for each state.

Provide smooth animation between algorithm steps.

Include

- Play
- Pause
- Next Step
- Previous Step
- Reset

Animation speed slider.

Sidebar Controls

Include

Graph

- Generate graph

- Upload graph
- Directed / Undirected
- Weighted / Unweighted

Algorithms

- Select algorithm
- Start node
- Goal node

Visualization

- Animation speed
 - Node size
 - Edge thickness
 - Show weights
 - Dark mode toggle
-

Metrics

After execution display

- Path found
 - Total distance
 - Number of visited nodes
 - Runtime (milliseconds)
 - Number of explored edges
 - Path length
 - Memory usage estimate
-

Algorithm Comparison

Allow users to run multiple algorithms on the same graph.

Display comparison table.

Columns

- Algorithm
- Runtime
- Nodes Visited
- Path Length
- Total Cost

Create bar charts comparing

- runtime
 - visited nodes
 - path cost
-

Educational Mode

Provide an explanation panel.

Describe

- How the selected algorithm works
 - Time complexity
 - Space complexity
 - Advantages
 - Disadvantages
 - Best use cases
-

Nice UI

Use Streamlit containers and columns.

Use metric cards.

Include expandable sections.

Add a clean landing page.

Provide helpful tooltips.

The application should feel polished.

Performance

Support graphs containing at least

- 300 nodes
- 1500 edges

without noticeable slowdown.

README

Write a professional README including

- Project overview
 - Features
 - Screenshots placeholder
 - Installation
 - Local execution
 - Streamlit deployment
 - Project architecture
 - Future improvements
-

Deployment

Include everything needed for deployment on Streamlit Community Cloud.

Generate

- requirements.txt
- .gitignore

The app should run using

```
streamlit run app.py
```

without modification.

Code Quality

- Modular architecture
- Meaningful variable names
- Docstrings
- Comments only where necessary
- No duplicated logic
- Error handling for invalid input
- Responsive layout

The finished project should resemble a polished portfolio application rather than a classroom assignment.